

PROJECT IMPULSE

Impulse

Single-Cycle CPU Internal Architecture Specification

ARIA-32 compatible 32-bit RISC processor | CPI = 1

DOCUMENT ID	IMPULSE-IAS-001
PROJECT	Impulse (Single-Cycle CPU)
REVISION	1.0
DATE	13 June 2026
STATUS	Released
ARCHITECTURE	ARIA-32 (ARM-inspired RISC ISA, 32-bit)
ISA VERSION	2.0
PIPELINE	Single-Cycle — no hazards, no stalls
DATA WIDTH	32-bit
IMEM / DMEM	4 KB / 4 KB (1024 x 32-bit words each)
TARGET HDL	Verilog 2001 / SystemVerilog (Vivado 2025)

1 Architecture Overview

ARIA-32 (ARM-inspired RISC ISA, 32-bit) is a fully custom, single-cycle, 32-bit RISC processor designed for teaching, research, and rapid algorithm prototyping on FPGA. The architecture draws inspiration from the ARM A32 programming model while using a simplified, clean encoding that fits in a single 32-bit instruction word. All instructions complete in exactly one clock cycle with no pipeline hazards or stalls.

Feature	Details
ISA Class	Custom 32-bit RISC (ARM-inspired)
Execution	Single-cycle — every instruction completes in 1 clock
Instruction Width	Fixed 32 bits
Instruction Formats	8 formats distinguished by bits [31:29]
Register File	16 × 32-bit (R0–R15; R13=SP, R14=LR)
Addressing	Register + 16-bit signed immediate offset
Branch Model	PC-relative word offset with 10 condition codes
Stack	Full-descending; SP initialised to 0xFFC; PUSH/POP
Interrupts	IRQ (vector 0x18) + FIQ (vector 0x1C), maskable
Memory	Harvard: IMEM 4 KB (APB write-only) / DMEM 4 KB
FPGA Target	Xilinx Artix-7 / Zynq-7 (Vivado 2025)

1.1 Pipeline Stage Mapping

Stage	RTL Location	Action
IF — Instruction Fetch	<code>inst_mem.v</code>	PC → IMEM; combinational read → <code>inst[31:0]</code>
ID — Instruction Decode	<code>cpu_top.v</code> , <code>control.v</code>	Extract <code>fmt/funct/Rd/Rs1/Rs2/imm16</code> ; generate all control signals
EX — Execute	<code>alu.v</code>	ALU operation; compute branch/BL targets; evaluate interrupt
MEM — Memory Access	<code>data_mem.v</code>	LDR read / STR write / PUSH-POP stack access
WB — Write Back	<code>cpu_top.v</code>	Select <code>WB_ALU / WB_MEM / WB_PC4</code> ; write register file; update PC & SR

2 Top-Level Block Diagram

The diagram below shows the major sub-blocks of the ARIA-32 datapath and their interconnections as implemented in `cpu_top.v`. All paths are combinational within a clock cycle except the sequential state elements (PC, SP, SR, register file, memories).

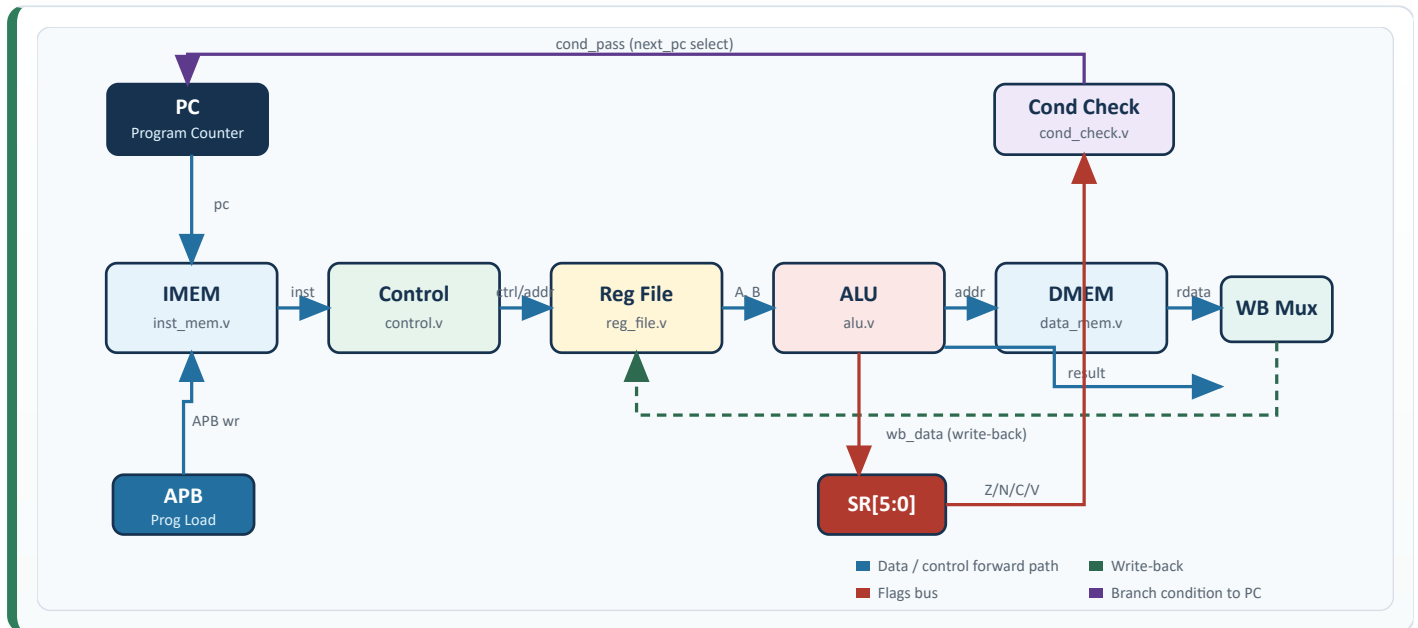


Figure 1 — ARIA-32 Single-Cycle Datapath

2.1 Signal Bus Summary

Signal / Bus	Width	Dir	Description
pc	32	Internal	Program Counter — byte address of current instruction
inst	32	IMEM → ID	Raw instruction word fetched from instruction memory
fmt	3	ID → ctrl	Instruction format selector bits [31:29]
funct	5	ID → ctrl	Function/opcode bits [28:24]
alu_op	4	ctrl → ALU	Internal ALU operation code (decoupled from ISA encoding)
alu_result	32	ALU → MEM/WB	ALU computed value; also used as memory address
mem_rdata	32	DMEM → WB	Data read from data memory (LDR / POP)
wb_data	32	WB → RF	Value written to register file (ALU, MEM, or PC+4)
sr[5:0]	6	Internal	Status Register: [5:F][4:I][3:V][2:C][1:N][0:Z]
cond_pass	1	cond_check → PC	Branch taken signal; only gates FMT_BRANCH
next_pc	32	Internal	Next PC mux output (PC+4 / branch / BL / BX / interrupt)

3 Programmer's Model

3.1 General-Purpose Register File

Reg	Alias	Role	Reset
R0	—	General purpose — conventional return value / scratch	0x00000000
R1–R12	—	General purpose	0x00000000
R13	SP	Stack Pointer — maintained in cpu_top, mirrored in RF; used by PUSH/POP	0x00000FFC
R14	LR	Link Register — receives PC+4 on BL; restored to PC by BX R14	0x00000000
R15	—	General purpose (PC is a dedicated register in cpu_top, not R15)	0x00000000

3.2 Status Register SR[5:0]

F 5	I 4	V 3	C 2	N 1	Z 0	unused 31:6
---------------	---------------	---------------	---------------	---------------	---------------	-----------------------

SR [only bits 5:0 are implemented; bits 31:6 are unused]

Bit	Name	Description	Set by
0	Z — Zero	Set when ALU result == 0	ADD SUB MUL AND OR XOR NOT LSL LSR ASR INC DEC CMP TST
1	N — Negative	Set when ALU result[31] == 1 (sign bit)	(same as Z)
2	C — Carry	Unsigned carry/borrow out; left-shift last ejected bit	ADD SUB INC DEC LSL LSR ASR
3	V — Overflow	Signed overflow	ADD SUB INC DEC
4	I — IRQ en	1=IRQ enabled; cleared on IRQ entry; restored by RETI	Reset=1; IRQ entry; RETI
5	F — FIQ en	1=FIQ enabled; cleared on FIQ entry; restored by RETI	Reset=1; FIQ entry; RETI

Note on MOV / MOVI: ALU_MOV and ALU_MOVB do *not* set flags_we. Use CMP or TST to set flags independently.

4 Instruction Encoding

All instructions are exactly 32 bits wide. Bits [31:29] select the instruction **format**; bits [28:24] select the specific **operation** within that format. The remaining bits carry register addresses and/or immediates.

fmt[31:29]	Binary	Format Name	Typical Use
000	0b000	R-Type (Register)	Rd = Rs1 OP Rs2
001	0b001	I-Type (Immediate)	Rd = Rs1 OP #imm16
010	0b010	Load	Rd = mem[Rb + #off16]
011	0b011	Store	mem[Rb + #off16] = Rs
100	0b100	Branch	if(COND) PC += #imm25 × 4
101	0b101	Unary	INC / DEC / PUSH / POP
110	0b110	Jump	BL #off20 / BX Rd
111	0b111	Control	NOP / RETI

4.1 R-Type [fmt=000] — Rd = Rs1 OP Rs2

fmt[2:0] 31:29	funct[4:0] 28:24	Rd[3:0] 23:20	Rs1[3:0] 19:16	Rs2[3:0] 15:12	unused[11:0] 11:0
--------------------------	----------------------------	-------------------------	--------------------------	--------------------------	-----------------------------

4.2 I-Type [fmt=001] — Rd = Rs1 OP #imm16

fmt 31:29	funct 28:24	Rd 23:20	Rs1 19:16	imm16[15:0] 15:0
---------------------	-----------------------	--------------------	---------------------	----------------------------

4.3 Load [fmt=010] — Rd = mem[Rb + #off16]

fmt 31:29	funct 28:24	Rd 23:20	Rb 19:16	off16[15:0] 15:0
---------------------	-----------------------	--------------------	--------------------	----------------------------

4.4 Store [fmt=011] — mem[Rb + #off16] = Rs

fmt 31:29	funct 28:24	Rs 23:20	Rb 19:16	off16[15:0] 15:0
---------------------	-----------------------	--------------------	--------------------	----------------------------

4.5 Branch [fmt=100] — if(COND) PC += #imm25 × 4

fmt 31:29	COND[3:0] 28:25	imm25[24:0] 24:0
---------------------	---------------------------	----------------------------

4.6 Unary [fmt=101] — INC/DEC/PUSH/POP Rd

fmt 31:29	funct 28:24	Rd 23:20	unused[19:0] 19:0
---------------------	-----------------------	--------------------	-----------------------------

4.7 Jump [fmt=110] — BL #off20 / BX Rd

fmt 31:29	funct 28:24	Rd/0[3:0] 23:20	imm20[19:0] 19:0
---------------------	-----------------------	---------------------------	----------------------------

4.8 Control [fmt=111] — NOP / RETI

fmt 31:29	funct 28:24	unused[23:0] 23:0
---------------------	-----------------------	-----------------------------

5 Instruction Set Reference

5.1 R-Type Instructions [fmt=000]

Mnemonic	funct	Encoding	Operation	Flags
ADD Rd, Rs1, Rs2	0x00	{000,00000,Rd,Rs1,Rs2,0x000}	Rd = Rs1 + Rs2	Z N C V
SUB Rd, Rs1, Rs2	0x01	{000,00001,Rd,Rs1,Rs2,0x000}	Rd = Rs1 − Rs2	Z N C V
MUL Rd, Rs1, Rs2	0x02	{000,00010,Rd,Rs1,Rs2,0x000}	Rd = Rs1 × Rs2 (lower 32 bits)	Z N
AND Rd, Rs1, Rs2	0x03	{000,00011,Rd,Rs1,Rs2,0x000}	Rd = Rs1 & Rs2	Z N
OR Rd, Rs1, Rs2	0x04	{000,00100,Rd,Rs1,Rs2,0x000}	Rd = Rs1 Rs2	Z N
XOR Rd, Rs1, Rs2	0x05	{000,00101,Rd,Rs1,Rs2,0x000}	Rd = Rs1 ^ Rs2	Z N
NOT Rd, Rs1	0x06	{000,00110,Rd,Rs1,-,0x000}	Rd = ~Rs1	Z N

Mnemonic	funct	Encoding	Operation	Flags
CMP Rs1, Rs2	0x07	{000,00111,-,Rs1,Rs2,0x000}	Rs1 – Rs2 (flags only; no write)	Z N C V
TST Rs1, Rs2	0x08	{000,01000,-,Rs1,Rs2,0x000}	Rs1 & Rs2 (flags only; no write)	Z N
LSL Rd, Rs1, Rs2	0x09	{000,01001,Rd,Rs1,Rs2,0x000}	Rd = Rs1 << Rs2[4:0]	Z N C
LSR Rd, Rs1, Rs2	0x0A	{000,01010,Rd,Rs1,Rs2,0x000}	Rd = Rs1 >> Rs2[4:0] (logical)	Z N C
ASR Rd, Rs1, Rs2	0x0B	{000,01011,Rd,Rs1,Rs2,0x000}	Rd = Rs1 >>> Rs2[4:0] (arithmetic)	Z N C
MOV Rd, Rs1	0x0C	{000,01100,Rd,Rs1,-,0x000}	Rd = Rs1 (no flag update)	–

5.2 I-Type Instructions [fmt=001]

The 16-bit immediate is **sign-extended** to 32 bits before use. For MOVI, imm16[15]=1 produces a negative value (e.g., 0xFFFF → 0xFFFFFFF = –1). The maximum safely positive MOVI value is 32767 (0x7FFF).

Mnemonic	funct	Operation	Flags
ADDI Rd, Rs1, #imm	0x00	Rd = Rs1 + SignExt(imm16)	Z N C V
SUBI Rd, Rs1, #imm	0x01	Rd = Rs1 – SignExt(imm16)	Z N C V
MOVI Rd, #imm	0x02	Rd = SignExt(imm16) (pass-B, no flags)	–
LSLI Rd, Rs1, #n	0x03	Rd = Rs1 << n (n = imm16[4:0])	Z N C
LSRI Rd, Rs1, #n	0x04	Rd = Rs1 >> n (logical, n = imm16[4:0])	Z N C
ASRI Rd, Rs1, #n	0x05	Rd = Rs1 >>> n (arithmetic, n = imm16[4:0])	Z N C

5.3 Memory Instructions [fmt=010 / 011]

Memory uses **word addressing**: `addr[11:2]` indexes into the 1024-word array (lower 2 bits are ignored). The effective byte address is `Rb + SignExt(off16)`.

Mnemonic	fmt	funct	Operation
LDR Rd, [Rb, #off]	fmt=010	0x00	Rd = mem[Rb + SignExt(off16)]
STR Rs, [Rb, #off]	fmt=011	0x00	mem[Rb + SignExt(off16)] = Rs

5.4 Branch Instructions [fmt=100]

Branch target: `PC + SignExt(imm25) × 4`. The offset is a **word offset from the current instruction** (not PC+4). Negative offset –N encodes as `225 – N`.

Mnemonic	COND[3:0]	Condition	Description
BEQ #off	0x1	Z==1	Branch if Equal / Zero
BNE #off	0x2	Z==0	Branch if Not Equal
BGT #off	0x3	~Z & N==V	Branch if Greater Than (signed)
BLT #off	0x4	N≠V	Branch if Less Than (signed)
BGE #off	0x5	N==V	Branch if Greater or Equal (signed)
BLE #off	0x6	Z N≠V	Branch if Less or Equal (signed)
BCS #off	0x7	C==1	Branch if Carry Set (unsigned ≥)

Mnemonic	COND[3:0]	Condition	Description
BCC #off	0x8	C==0	Branch if Carry Clear (unsigned <)
B #off	0xE	Always	Unconditional branch (COND_AL)
(NOP) #off	0x0	Never	Never-taken (COND_NV) — acts as NOP

5.5 Unary Instructions [fmt=101]

Unary instructions use `use_rd_src=1`, routing the *Rd* field into Register File Port A. PUSH/POP interact with SP which is maintained in `cpu_top`.

Mnemonic	funcnt	Operation	Flags
INC Rd	0x00	Rd = Rd + 1	Z N C V
DEC Rd	0x01	Rd = Rd − 1	Z N C V
PUSH Rd	0x02	mem[SP−4] = Rd; SP −= 4 (full-descending)	—
POP Rd	0x03	Rd = mem[SP]; SP += 4	—

5.6 Jump Instructions [fmt=110]

Mnemonic	funcnt	Operation
BL #off20	0x00	LR (R14) = PC+4; PC = PC + SignExt(off20) × 4 off20 is a word offset from current instruction.
BX Rd	0x01	PC = Rd. Used for subroutine return (BX R14).

5.7 Control Instructions [fmt=111]

Mnemonic	funcnt	Operation
NOP	0x00	No operation; PC advances normally
RETI	0x01	Return from interrupt: PC = LR_irq/LR_fiq; SR restored from SPSR; mode → SYS

6 Condition Codes

The 4-bit COND field in branch instructions [28:25] selects one of 10 implemented conditions evaluated by `cond_check.v` against the SR flags. COND_NV (0x0) always evaluates to false.

Code	Define	Condition	Expression	Mnemonic
0x0	COND_NV	Never	0 (constant false)	NOP / reserved
0x1	COND_EQ	Equal / Zero	Z	BEQ
0x2	COND_NE	Not Equal	~Z	BNE
0x3	COND_GT	Greater Than	~Z & (N==V)	BGT
0x4	COND_LT	Less Than	N != V	BLT
0x5	COND_GE	Greater or Equal	N == V	BGE
0x6	COND_LE	Less or Equal	Z (N != V)	BLE
0x7	COND_CS	Carry Set	C	BCS (unsigned >=)

Code	Define	Condition	Expression	Mnemonic
0x8	COND_CC	Carry Clear	~C	BCC (unsigned <)
0x9–0xD	–	Reserved	–	(evaluate as false)
0xE	COND_AL	Always	1 (constant true)	B (unconditional)
0xF	–	Reserved	–	(evaluate as false)

7 Sub-Module Descriptions

7.1 cpu_top.v — Top-Level Integration

The top module instantiates all sub-blocks and wires the five-stage combinational datapath. All sequential state (PC, SP, SR, cpu_mode, saved LR/SPSR registers) is updated on the rising edge of `clk`. The APB bus is clocked by the independent `pcclk`.

Signal	Dir	W	Description
clk	in	1	CPU execution clock
pcclk	in	1	APB program-load clock (independent of clk)
rst_n	in	1	Active-low CPU reset: clears PC, SP, SR, RF, mode registers
prst_n	in	1	Active-low APB reset: gates APB transactions, does NOT clear IMEM
irq	in	1	Maskable interrupt request (requires SR[I]=1)
fiq	in	1	Fast interrupt request (requires SR[F]=1, higher priority than IRQ)
paddr	in	32	APB address (byte address → IMEM word = paddr[11:2])
psel	in	1	APB slave select
penable	in	1	APB enable / ACCESS phase strobe
pwrite	in	1	APB direction: 1=write, 0=read
pwdata	in	32	APB write data (instruction word)
prdata	out	32	APB read data (combinational)
pready	out	1	APB ready: prst_n & psel & penable (always single-cycle)
pslverr	out	1	APB slave error: always 0 (no error conditions)

7.2 inst_mem.v — Instruction Memory (4 KB, APB Completer)

A 1024-word × 32-bit synchronous-write / asynchronous-read RAM that serves dual roles: it is loaded by the host via APB writes (with `rst_n=0, prst_n=1`) and then read combinatorially by the CPU fetch stage. Memory contents are preserved across `prst_n` de-assertion; CPU reset (`rst_n`) does not clear IMEM.

Loading sequence: Assert `rst_n=0` (CPU halted), deassert `prst_n=1` (APB active). Write each instruction word via APB. Then assert `rst_n=1` to start execution. The CPU will fetch from PC=0x00000000 on the first cycle.

7.3 control.v — Two-Level Instruction Decoder

A purely combinational two-level decoder: the outer `case(fmt)` selects the instruction class; the inner `case(funcnt)` selects the specific operation. All outputs default to safe inactive states and are overridden only for recognised encodings.

Output Signal	W	Function
use_imm	1	Route sign-extended imm16 to ALU port B instead of Rs2

Output Signal	W	Function
use_rd_src	1	Route Rd field to RF port A address (used by UNARY, BX)
reg_write	1	Enable register file write-back
mem_read	1	Indicates a memory load (LDR, POP)
mem_write	1	Enable data memory write (STR, PUSH)
wb_sel	2	WB mux: 00=ALU result, 01=mem_rdata, 10=PC+4 (BL)
branch	1	Instruction is a conditional branch
bl_op	1	BL operation: save PC+4 to LR, jump to BL target
bx_op	1	BX operation: jump to register value
push_op	1	PUSH: mem[SP-4]=Rd; SP-=4
pop_op	1	POP: Rd=mem[SP]; SP+=4
reti_op	1	Return from interrupt: restore PC and SR from saved copies
cmp_tst_op	1	CMP/TST: run ALU, update flags, suppress register write
alu_op	4	Internal ALU opcode sent to alu.v (see ALU operations table)

7.4 reg_file.v — 16 × 32-bit Register File

Two asynchronous read ports (Port A and Port B) and one synchronous write port. Port A normally reads Rs1; for UNARY and BX instructions it reads Rd (selected by use_rd_src in cpu_top). Port B normally reads Rs2; for STR instructions it reads the source register (mapped via rd_addr). All registers reset to zero on rst_n assertion.

7.5 alu.v — Arithmetic and Logic Unit

Fully combinational. Receives a 4-bit internal alu_op code from the control unit (not the raw ISA funct field). Carries a 33-bit internal accumulator tmp[32:0] for carry detection. Z and N flags are always derived from result when flags_we=1; C and V are only meaningful for arithmetic/shift operations.

alu_op	Define	Operation	Flags	Used by
0x0	ALU_ADD	a + b	Z N C V	ADD ADDI INC (address calc)
0x1	ALU_SUB	a – b	Z N C V	SUB SUBI DEC CMP
0x2	ALU_MUL	a × b (32-bit)	Z N	MUL
0x3	ALU_AND	a & b	Z N	AND TST
0x4	ALU_OR	a b	Z N	OR
0x5	ALU_XOR	a ^ b	Z N	XOR
0x6	ALU_NOT	~a	Z N	NOT
0x7	ALU_LSL	a << b[4:0]	Z N C	LSL LSLI
0x8	ALU_LSR	a >> b[4:0] (logical)	Z N C	LSR LSRI
0x9	ALU_ASR	\$signed(a) >>> b[4:0]	Z N C	ASR ASRI
0xA	ALU_MOV	a (pass-through)	–	MOV
0xB	ALU_MOVB	b (imm32)	–	MOVI
0xC	ALU_INC	a + 1	Z N C V	INC

alu_op	Define	Operation	Flags	Used by
0xD	ALU_DEC	$a - 1$	Z N C V	DEC

7.6 data_mem.v — Data Memory (4 KB)

A 1024-word × 32-bit synchronous-write / asynchronous-read RAM. The effective word index is `addr[11:2]` (lower 2 bits ignored — word-aligned access). The read port is always-active (no read-enable); the write port is gated by `we`. DMEM is accessible only by the CPU datapath — there is no APB read/write port for data memory (backdoor testbench access via `dut.u_dmem.mem[N]` requires `-debug all` elaboration).

Memory map: Byte address 0x000–0xFFFF (4 KB). Word 0 = byte 0x000; Word 1023 = byte 0xFFC. SP initialises to 0xFFC (top of DMEM), growing downward.

7.7 cond_check.v — Condition Code Evaluator

Purely combinational. Evaluates a 4-bit COND field from the branch instruction against the four SR flags (Z, N, C, V). The output `pass` is 1 only when the condition is satisfied. It gates the PC branch target in `cpu_top` only for FMT_BRANCH instructions; all other instruction formats execute unconditionally.

8 APB Programming Interface

Programs are loaded into IMEM via the AMBA APB bus. The CPU must be held in reset (`rst_n=0`) during loading so the fetch stage does not interfere. The APB clock (`pcclk`) is independent of the CPU clock (`clk`).

8.1 Load Sequence

Step	Signal State	Action
1	<code>rst_n=0</code> , <code>prst_n=1</code>	CPU halted; APB enabled
2	APB SETUP phase	Set <code>paddr</code> , <code>psel=1</code> , <code>pwrite=1</code> , <code>pwdata=instruction</code>
3	APB ACCESS phase	Assert <code>penable=1</code> ; <code>pready</code> goes high; word written to IMEM
4	Repeat for each word	Increment <code>paddr</code> by 4 per instruction
5	<code>rst_n=1</code>	Release CPU reset; execution begins at PC=0x00000000

8.2 Address Map

IMEM byte address = `instruction_word_index × 4`. Vivado testbenches write using `apb_write(addr, data)` where `addr` is the byte address. The boot vector at word 0 (byte 0x000) must contain a branch to the program entry point, and interrupt vectors at 0x018 (IRQ) and 0x01C (FIQ) must contain valid RETI instructions.

Byte Addr	Word	Convention
0x000	0	Boot vector: B AL #entry_offset (jump to main program)
0x018	6	IRQ vector: RETI (or ISR entry)
0x01C	7	FIQ vector: RETI (or ISR entry)
0x080	32	Conventional program start (B AL #32 from boot)
0x100	64	Conventional DMEM data base (byte 0x100 = word 64)
0xFFC	1023	SP initial value (top of DMEM, full-descending stack)

9 Interrupt Architecture

ARIA-32 supports two interrupt levels: FIQ (Fast Interrupt Request, higher priority) and IRQ (Interrupt Request). Interrupts are only accepted when the CPU is in SYS mode and the corresponding enable flag (SR[F] or SR[I]) is set. FIQ and IRQ are edge-triggered and evaluated every clock cycle.

Parameter	IRQ	FIQ
Vector address	0x00000018	0x0000001C
Enable flag	SR[I] (bit 4)	SR[F] (bit 5)
Saved PC	lr_irq = PC+4	lr_fiq = PC+4
Saved SR	spsr_irq = SR	spsr_fiq = SR
On entry	SR[I]←0; mode←IRQ	SR[F]←0; SR[I]←0; mode←FIQ
Return	RETI: PC←lr_irq; SR←spsr_irq; mode←SYS	RETI: PC←lr_fiq; SR←spsr_fiq; mode←SYS
Priority	Lower	Higher (evaluated first)
Accepted in mode	SYS only	SYS only

Interrupt sequence (single-cycle): When an interrupt is accepted, the next_pc mux routes to the vector address on the *same cycle* as the current instruction. The interrupted instruction still completes write-back before the vector is fetched, so the return address (PC+4) correctly points to the instruction after the interrupted one.